

GraphQL på Datafordeleren

Martin Jensen, Klimadatastyrelsen

Margit Kildevang, Klimadatastyrelsen

Morten Fuglsang, Septima

Hvorfor ?



Nuværende Datafordeler

- ÷ Nuværende REST-tjenester med forskellig logik og struktur fra register til register
- ÷ Komplekst for registre at specificere REST-tjenester via DLS – og høj time-to-market for implementering heraf
- ÷ Meget manuelt arbejde ifm. implementering af nye og ændrede REST-tjenester
- ÷ Mange data, der ellers er på Datafordeleren, er ikke udstillet via REST-tjenester
- ÷ Udstillingens output skemaer følger ikke altid grunddatamodellen



Moderniserede Datafordeler

- + GraphQL-udstilling for alle entiteter, som anvendere har adgang til
- + Samme logik og struktur for opslag af data, uanset register
- + Ingen register-specifikation (DLS) af udstillings-skemaer, tjenesteparametre eller tjenestelogik
- + Autogenerering af implementering af nye og ændrede GraphQL-tjenester, med minimalt manuelt arbejde
- + Anvendere kan nøjes med at hente de data der er brug for
- ! Anvendere skal (som hidtil) forholde til sammenhæng mellem data ud fra grunddatamodellen

Kort intro til GraphQL

Hvorfor GraphQL

Udfordringer med REST:

- Overfetching og underfetching af data
- Mange API-kald for at hente relaterede data

Hvad er en graf ?

En graf består af:

- **Noder (nodes):** Objekter som fx `Bruger` , `Adresse` , `Postnummer`
- **Kanter (edges):** Relationer mellem objekterne (fx en bruger har en adresse)
- **Felter (fields):** Informationer på hver node

En GraphQL-query følger forbindelserne i grafen
Du spørger på tværs af datarelationer – fleksibelt

Hvad er en query ?

Query: Nøgleordet for en læseoperation

Vi beder om et user-objekt med id =
"123"

Vi specificerer præcis hvilke felter vi vil
have: id, name, email

```
query {  
  user(id: "123") {  
    id  
    name  
    email  
  }  
}
```

Hvordan sender man en forespørgsel ?

GraphQL bruger som regel HTTP POST (kan også være GET)

Body'en indeholder query som tekst (JSON)

```
POST /graphql
```

```
Content-Type: application/json
```

```
{  
  "query": "query { user(id: \"123\") { name email } }"  
}
```



REST REQUEST

GET https://sample.com/person/1

REST JSON

```
{
  "firstName": "John",
  "middleName": "Andrew",
  "lastName": "Smith",
  "email": "jas1992@gmail.com",
  "relationshipStatus": "single"
}
```

VS



GraphQL

GRAPHQL QUERY

```
{
  person {
    firstName
    lastName
  }
}
```

GRAPHQL JSON

```
{
  "data": {
    "person": {
      "firstName": "John",
      "lastName": "Smith",
    }
  }
}
```


GraphQL schemas

Et GraphQL-schema definerer de typer og felter, du kan spørge om i API'et.

Schemaet fungerer som:

- En **kontrakt** mellem klient og server
- En **struktur** over alle data og deres relationer
- En **validering** af dine queries

Et schema består af:

- **type** – definerer objekttyper (f.eks. **Adresse** , **Vejstykke**)
- **Query** – definerer tilgængelige læseoperationer
- **Mutation** – definerer skrive-/opdateringsoperationer (hvis tilladt)

Eksempel fra Datafordeleren

```
query {  
  BBR_Bygning( # Entitet  
    where: { # Standardfiltre  
      kommunekode: { eq: "0550" }  
      virkningsaktoer: { startsWith: "Konvertering" }  
    }  
    virkningstid: "2024-11-12T14:41:33Z" # Bitemporalt filter  
    first: 10 # Paging  
  ) {  
    pageInfo { # Paginginformation  
      startCursor  
      endCursor  
      hasNextPage  
      hasPreviousPage  
    }  
    nodes {  
      id_lokalId  
      kommunekode  
      virkningsaktoer  
    }  
  }  
}
```

- **Entitet:** Hver query skal specificere hvilken entitet den omhandler. Eksemplet ovenfor viser en query for entiteten BBR_Bygning fra BBR og specifikt kun for 3 kolonner (angivet i "nodes"): id_lokalId, kommunekode og virkningsaktoer.
- **Endepunkt:** En query sendes til et GraphQL-endepunkt som et GET- eller POSTrequest.
- **Standardfiltre:** En query kan indeholde standardfiltre der filtrerer data. Query'en ovenfor indeholder to standardfiltre: et lighedsfilter (eq: "0550") på kommunekode-kolonnen og et tekstfilter (startsWith: "Konvertering") på virkningsaktoer-kolonnen.

- **Geometriske filtre:** En query kan indeholde geometriske filtre, der filtrerer data på baggrund af geometri.
- **Bitemporale filtre:** En query kan også indeholde bitemporale filtre. Query'en oven for indeholder et bitemporalt point-in-time-filter (virkningstid: "2024-11-12T14:41:33Z") på virkningstidspunktet.
- **Paging :** En query kan også definere hvordan data pagineres. Query'en oven for returnerer de første 10 rækker (first: 10) givet filtreringen samt paginginformation for resultatsættet (angivet i "pageInfo").

Hvordan får man adgang ?

- Det er nødvendigt at etablere en ny type adgang - det er ikke længere nok med en webbruger og en tjenestebruger.
- Hvis man skal have adgang til frie data, er en token nok, men den skal oprettes

Administration

Log på Datafordelerens Administration og opret de IT-systemer, der skal have adgang til at hente data via Fildownload (HTTPS) og GraphQL.

DATAFORDELER ADMINISTRATION

Bemærk at hvert testmiljø har egen Administration.

TEST 03

TEST 04

TEST 06

Bemærk: Du skal ansøge i Administration for at få adgang til beskyttede data. Under hvert IT-system kan du oprette API-keys eller OAuth Shared Secrets, som du skal indsætte i URL for at hente data via HTTPS. Du kan også oprette og administrere andre anvendere, der skal have adgang til IT-systemer eller Administration.

Lad os hente nogle schemaer

`graphql.datafordeler.dk/DAR/v1/schema?apiKey=API-KEY`

`graphql.datafordeler.dk/BBR/v1/schema?apiKey=API-KEY`

`graphql.datafordeler.dk/MAT/v1/schema?apiKey=API-KEY`

Vi starter med at kigge nærmere på DAR schemaet...

Elementer

- Metadata
- Query types
- Filter inputs
- Spatiale filtre

```

schema {
  query: Query
}

interface SpatialInterfaceType {
  "The OGC geometry type of the geometry."
  type: SpatialGeometryNameEnumType!
  "The coordinate reference system of the geometry."
  crs: Int!
  "The dimensions used by the geometry."
  dimension: SpatialDimensionEnumType!
  "The WKT representation of the geometry"
  wkt: String
}

"""
Grunddatamodel info:
name: Adresse
comment (da): <memo>#NOTES#en sammensat betegnelse, som udpeger o
definition (da): <memo>#NOTES#en struktureret betegnelse som angi
example (da): <memo>#NOTES#adressebetegnelse: Gimles Allé 14, 230
historikmodel: bitemporalitet#NOTES#Values: registreringshistorik
legalSource: https://www.retsinformation.dk/eli/lta/2018/271
prefLabel (da): Adresse
URI: https://data.gov.dk/model/profile/dar/Adresse
EAID: EAID_6043F4D9_C4CA_44d0_BE88_027166A8B008
"""

type DAR_Adresse {
  """
  Grunddatamodel info:
  type: CharacterString
  multiplicity: 1
  URI: https://data.gov.dk/model/profile/dar/adressebetegnelse
  """
}

```

Query types

```
"Query type for DAR - Danmarks Adresseregister"
```

```
type Query {
```

```
  DAR_Adressepunkt("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that
```

```
  DAR_Adresse("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that cor
```

```
  DAR_Husnummer("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that c
```

```
  DAR_NavngivenVejKommunedel("Returns the first _n_ elements from the list." first: Int "Returns the elements in th
```

```
  DAR_NavngivenVejPostnummer("Returns the first _n_ elements from the list." first: Int "Returns the elements in th
```

```
  DAR_NavngivenVejSupplerendeBynavn("Returns the first _n_ elements from the list." first: Int "Returns the element
```

```
  DAR_NavngivenVej("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list th
```

```
  DAR_Postnummer("Returns the first _n_ elements from the list." first: Int "Returns the elements in the list that
```

```
  DAR_SupplerendeBynavn("Returns the first _n_ elements from the list." first: Int "Returns the elements in the li
```

```
}
```

Filter inputs

```
input DAR_AdresseFilterInput {  
  and: [DAR_AdresseFilterInput!]  
  adressebetegnelse: DafSearchableStringOperationFilterInput  
  bygning: DafStringOperationFilterInput  
  datafordelerOpdateringstid: DafDateTimeOperationFilterInput  
  doerbetegnelse: DafStringOperationFilterInput  
  doerpunkt: DafStringOperationFilterInput  
  etagebetegnelse: DafStringOperationFilterInput  
  husnummer: DafStringOperationFilterInput  
  id_lokalId: DafStringOperationFilterInput  
  registreringFra: DafDateTimeOperationFilterInput  
  registreringsaktoer: DafStringOperationFilterInput  
  registreringTil: DafDateTimeOperationFilterInput  
  status: DafStringOperationFilterInput  
  virkningFra: DafDateTimeOperationFilterInput  
  virkningsaktoer: DafStringOperationFilterInput  
  virkningTil: DafDateTimeOperationFilterInput  
}
```

Spatial Filters

```
input SpatialFilterInput {  
  and: [SpatialFilterInput!]  
  or: [SpatialFilterInput!]  
  contains: SpatialContainsOperationFilterInput @cost(weight: "5000")  
  covers: SpatialCoversOperationFilterInput @cost(weight: "5000")  
  crosses: SpatialCrossesOperationFilterInput @cost(weight: "5000")  
  intersects: SpatialIntersectsOperationFilterInput @cost(weight: "5000")  
  within: SpatialWithinOperationFilterInput @cost(weight: "5000")  
  overlaps: SpatialOverlapsOperationFilterInput @cost(weight: "5000")  
  touches: SpatialTouchesOperationFilterInput @cost(weight: "5000")  
  coveredBy: SpatialCoveredByOperationFilterInput @cost(weight: "5000")  
}
```


Metadata

```
type DAR_Adresse {  
    ""  
    Grunddatamodel info:  
    type: CharacterString  
    multiplicity: 1  
    URI: https://data.gov.dk/model/profile/dar/adressebetegnelse  
    prefLabel (da): adressebetegnelse  
    definition (da): en læsbar struktur, som identificerer adressen fuldstændigt  
    legalSource: https://www.retsinformation.dk/eli/lta/2018/271#P12  
    ""  
    adressebetegnelse: String @cost(weight: "1")  
    bygning: String @cost(weight: "1")  
    datafordelerOpdateringstid: DafDateTime @cost(weight: "1")  
    ""  
    Grunddatamodel info:  
    type: CharacterString  
    multiplicity: 0..1  
    URI: https://data.gov.dk/model/profile/dar/dørbetegnelse  
    prefLabel (da): dørbetegnelse  
    definition (da): betegnelse, som angiver den adgangsdør e.l. som adressen identificerer  
    legalSource: https://www.retsinformation.dk/eli/lta/2018/271#P22 || https://www.retsinforma  
    ""  
}
```

Basis request

Endpoint : `graphql.datafordeler.dk/DAR/v1?apiKey=API-KEY`

```
query {
  DAR_Adresse(
    registreringstid: "2025-06-04T00:00:00Z"
    virkningstid: "2025-06-04T00:00:00Z"
    first: 10
  ) {
    nodes {
      adressebetegnelse
      id_lokalId
      husnummer
      doerbetegnelse
      etagebetegnelse
      status
      registreringFra
      virkningFra
    }
  }
}
```

Hands on eksempler på GraphQL forespørgsler